Codificación Binaria x86 (Generación de Código)

Beltran Salazar Norberto Rafael 23170707 1:00 – 2:00

Objetivo: Representar en código máquina x86 de 16 bits las instrucciones que genera la gramática para asignaciones, operaciones aritméticas, comparaciones y saltos.

Base: Gramática BNF y generador de código intermedio del Lenguaje Genesis, que emite instrucciones 8086/MASM con registros de 16 bits (AX, BX, DX, etc.).

1. MOV: carga de memoria al registro acumulador

- Instrucción: **MOV AX, [num]**
- Hex: **A1 00 00** Binario: **10100001 00000000 00000000**
- Desglose:
  - **A1:** opcode optimizado de Intel exclusivo para mover una palabra de 16 bits desde una dirección de memoria directa hacia el acumulador AX.
  - **00 00:** dirección de memoria (offset 0x0000) en formato Little Endian.
  - No requiere byte ModR/M.

2. LEA: carga de dirección efectiva

- Instrucción: **LEA DX, [total]**
- Hex: **8D 16 04 00** Binario: **10001101 00010110 00000100 00000000**
- Desglose:
  - **8D:** opcode que indica la operación LEA (Load Effective Address) hacia un registro de 16 bits.
  - **16 = 00 | 010 | 110** (ModR/M): MOD=00 y R/M=110 indican desplazamiento directo de 16 bits; REG=010 identifica al registro destino DX.
  - **04 00:** dirección de memoria (offset 0x0004) en formato Little Endian.

3. ADD: suma de memoria al acumulador (@)

- Instrucción: **ADD AX, [num2]**
- Hex: **03 06 02 00** Binario: **00000011 00000110 00000010 00000000**
- Desglose:
  - **03:** opcode que indica la operación ADD hacia un registro de 16 bits.
  - **06 = 00 | 000 | 110** (ModR/M): MOD=00 y R/M=110 indican memoria directa de 16 bits; REG=000 selecciona al registro destino AX.
  - **02 00:** dirección de memoria (offset 0x0002) en formato Little Endian.

4. SUB: resta de memoria al acumulador (#)

- Instrucción: **SUB AX, [num2]**
- Hex: **2B 06 02 00** Binario: **00101011 00000110 00000010 00000000**
- Desglose:
  - **2B:** opcode SUB r16 ← r/m16 (resta hacia un registro de 16 bits).
  - **06 = 00 | 000 | 110** (ModR/M): misma forma que ADD; REG=000 → AX.
  - **02 00:** offset 0x0002 en Little Endian.

5. CMP: comparación (condiciones if / while / for)

- Instrucción: **CMP AX, 0**
- Hex: **3D 00 00** Binario: **00111101 00000000 00000000**
- Desglose:
  - **3D:** opcode optimizado CMP AX, imm16; actualiza las banderas ZF, SF, CF y OF usadas por los saltos condicionales.
  - **00 00:** inmediato 0 en formato Little Endian.
  - No requiere byte ModR/M.

6. PUSH: guardar registro en la pila

- Instrucción: **PUSH AX**
- Hex: **50** Binario: **01010000**
- Desglose:
  - **50:** opcode de 1 byte donde los primeros 5 bits (01010) indican la operación PUSH de registro de 16 bits, y los últimos 3 bits (000) identifican al registro AX.

7. POP: recuperar registro desde la pila

- Instrucción: **POP AX**
- Hex: **58** Binario: **01011000**
- Desglose:
  - **58:** opcode de 1 byte (58 + rw). Bits 01011 = POP de registro de 16 bits; bits 000 = registro AX.

8. JMP: salto incondicional

- Instrucción: **JMP WHILE_1** (ejemplo rel8 = −20)
- Hex: **EB EC** Binario: **11101011 11101100**
- Desglose:
  - **EB:** opcode para salto incondicional corto.
  - **EC:** desplazamiento relativo de 1 byte en complemento a dos (−20 en decimal), que retrocede el puntero de instrucción (IP) al inicio del bucle.
  - No usa ModR/M.

9. JE / JNE: saltos condicionales

- Instrucción: **JE L2** (rel8 = +10) / **JNE L2** (rel8 = +8)
- Hex: **74 0A** / **75 08** Binario: **01110100 00001010** / **01110101 00001000**
- Desglose:
  - **74:** opcode JE/JZ (salta si ZF = 1). Usado cuando == es falso → JNE; o al comparar AX con 0.
  - **75:** opcode JNE/JNZ (salta si ZF = 0). El generador emite JNE cuando el operador relacional es == (saltar si la condición es falsa).
  - El segundo byte es el desplazamiento relativo (rel8) respecto al siguiente IP.

10. INT: servicio de salida DOS

- Instrucción: **INT 21h**
- Hex: **CD 21** Binario: **11001101 00100001**
- Desglose:
  - **CD:** opcode de interrupción software con inmediato de un byte.
  - **21:** número de interrupción 21h de DOS; AH selecciona el servicio de impresión (09h/02h), lectura (01h) o terminación.

Estas representaciones corresponden a los bytes que el backend de generación de código (o un ensamblador MASM) escribiría en el archivo objeto para cada instrucción emitida a partir del código intermedio de la gramática.